Product Requirements Document

Product: Simple E-Commerce Store
Version: 1.0
Date: 2026-11-06
Status: Draft — ready for implementation

1. Overview

A small, self-contained online store where visitors can browse products, read product details, add items to a cart, create an account, and place an order. A staff user can manage the product catalog and view incoming orders through an admin panel.

The goal is a working, understandable store — not a scalable commerce platform. Every decision in this document favors the simplest option that still produces a complete, functional site.

1.1 Goals

#	Goal
G1	A visitor can browse the product catalog and view any product's details.
G2	A visitor can add products to a cart, change quantities, and remove items.
G3	A visitor can register an account and log in and out.
G4	A logged-in user can complete checkout and see the order they placed.
G5	A staff user can add, edit, and delete products, and view all orders.

1.2 Non-Goals

These are explicitly **out of scope** for version 1.0. They are listed so that scope stays stable during implementation.

- Real payment processing (no card data is ever collected or stored)
- Product reviews, ratings, or comments
- Wishlists, favorites, or carts saved across devices
- Discount codes, coupons, or promotions
- Shipping-rate calculation or carrier integration
- Email sending of any kind (no order confirmations, no password-reset emails)
- Product search or filtering beyond category
- Multiple images per product
- Product variants (size, color)
- Inventory reservation beyond a simple stock check at checkout
- Order cancellation or returns by the customer
- Mobile app, public REST API, or admin analytics dashboard

2. Technology Stack

Layer	Choice	Reason
Backend	Python 3.12 + Django 6.0	Built-in authentication, admin panel, ORM, and CSRF protection remove most of the code this project would otherwise need.
Frontend	Django templates (HTML), plain CSS, vanilla JavaScript	Server-rendered pages. No build step, no bundler, no frontend framework.
Database	PostgreSQL 14+	Required by the brief. Accessed through the Django ORM via `psycopg[binary]`.
Admin	Django Admin (built in)	A product and order management UI with almost no custom code.
Payment	None — mock checkout	Orders are placed with status `pending` and settled as cash on delivery.

2.1 Why Django templates and not a single-page app

The brief asks for HTML, CSS, and JavaScript on the frontend. Django templates satisfy that: each page is HTML served by Django, styled with one CSS file, with JavaScript added only where a page needs interactivity without a reload (cart quantity updates, the cart item count in the header). This keeps the whole store in one codebase and avoids building a separate API layer.

---

3. User Roles

Role	Description	Capabilities
Guest	Not logged in.	Browse products, view product details, add to cart, register, log in.
Customer	Registered and logged in.	Everything a guest can do, plus checkout, view own order history, log out.
Staff	Created with `createsuperuser`.	Access the Django admin panel: manage products and categories, view and update orders.

Staff accounts are not created through the website. There is no public staff registration.

---

4. Data Model

Five tables. Django's built-in `auth_user` table provides users, so no custom user model is needed.

4.1 User (Django built-in `auth.User`)

Used as-is. Relevant fields: `id`, `username`, `email`, `password` (hashed), `first_name`, `last_name`, `is_staff`, `date_joined`.

Login is by username. Keeping the default user model means there is nothing to migrate later if the project stays at this scope.

4.2 Category

Field	Type	Notes
`id`	serial	Primary key
`name`	varchar(100)	Unique. Example: "Electronics"
`slug`	varchar(100)	Unique. URL-friendly, e.g. `electronics`

4.3 Product

Field	Type	Notes
`id`	serial	Primary key
`name`	varchar(200)	
`slug`	varchar(200)	Unique. Used in the product detail URL.
`description`	text	Shown on the product detail page
`price`	numeric(10,2)	Always a `Decimal`, never a float
`stock`	integer	Default 0. A product with stock 0 shows as "Out of stock" and cannot be added to the cart.
`image`	image upload	One image, stored under `media/products/`. Optional; a placeholder is shown when empty.
`category_id`	FK to Category	`ON DELETE PROTECT`, so deleting a category that still has products fails loudly instead of silently removing them
`is_active`	boolean	Default true. False hides the product from the storefront but keeps it readable on past orders.
`created_at`	timestamptz	Set automatically on insert

4.4 Order

Field	Type	Notes
`id`	serial	Primary key, also the order number shown to the customer
`user_id`	FK to User	`ON DELETE PROTECT`. Checkout requires login, so this is never null.
`full_name`	varchar(150)	Shipping recipient
`email`	varchar(254)	Contact address for this order
`phone`	varchar(30)	
`address`	text	Street address
`city`	varchar(100)	
`postal_code`	varchar(20)	
`total`	numeric(10,2)	Sum of all line totals, written once at checkout
`status`	varchar(20)	One of `pending`, `paid`, `shipped`, `delivered`, `cancelled`. Starts at `pending`.
`created_at`	timestamptz	Set automatically on insert

4.5 OrderItem

Field	Type	Notes
`id`	serial	Primary key
`order_id`	FK to Order	`ON DELETE CASCADE`
`product_id`	FK to Product	`ON DELETE PROTECT`
`product_name`	varchar(200)	Copied from the product at checkout
`price`	numeric(10,2)	Copied from the product at checkout
`quantity`	integer	Minimum 1

Why `product_name` and `price` are copied rather than read through the foreign key: if a product is later renamed or repriced, past orders must still show what the customer actually bought and what they actually paid. Without the copy, every historical order and total silently changes whenever staff edit a price.

4.6 Cart

The cart is **not** a database table. It lives in the Django session as a dictionary keyed by product ID:

```
request.session["cart"] = {
    "12": {"quantity": 2},
    "7": {"quantity": 1},
}
```

Name and price are read live from the `Product` table whenever the cart is displayed, so the cart always reflects current prices. A session cart works for guests and survives login, which keeps add-to-cart open to visitors who have not registered yet.

---

5. Pages and Features

5.1 Product list — `/`

The storefront home page.

- Active products in a responsive grid: image, name, price, and an "Add to cart" button.
- A category bar filters the grid via `/category/<slug>/`.
- Pagination at 12 products per page.
- Out-of-stock products stay visible but show "Out of stock" in place of the add-to-cart button.

5.2 Product detail — `/product/<slug>/`

- Large image, name, price, full description, category, and stock status.
- A quantity input (minimum 1, maximum the current stock) and an "Add to cart" button.
- A "Back to products" link.

5.3 Cart — `/cart/`

- A table of every line: image thumbnail, name, unit price, quantity, line total, and a remove button.
- Quantity can be changed inline; the line total and cart total update without a full page reload (vanilla JavaScript `fetch`).
- Cart total below the table.
- A "Proceed to checkout" button. For a guest this redirects to login, then returns to checkout.
- An empty cart shows a message and a link back to the product list.
- The header shows a live item count on every page.

5.4 Registration — `/accounts/register/`

- Fields: username, email, password, password confirmation.
- Built on Django's `UserCreationForm` plus a required email field.
- Validation: username unique, email format valid, the two passwords match, and the password passes Django's default validators (at least 8 characters, not entirely numeric, not a common password).
- On success the user is logged in automatically and sent to the product list.

5.5 Login and logout — `/accounts/login/`, `/accounts/logout/`

- Django's built-in `LoginView` and `LogoutView` with custom templates.
- A failed login shows one non-specific error, "Invalid username or password", so the page does not reveal whether a username exists.
- Logout is a POST. A GET logout lets a link or image on another site log the user out without their intent.
- Logging in does not clear the session cart.

5.6 Checkout — `/checkout/`

Login required. A guest arriving here is redirected to login and returned afterward.

- A shipping form: full name, email, phone, address, city, postal code. Name and email are prefilled from the account where available.
- An order summary listing every cart line and the total.
- A "Place order" button.

On submission, all of the following happens inside a single database transaction:

1. Re-read every cart product from the database and re-check its stock.
2. If any line exceeds available stock, abort the whole transaction and return to the cart with an explanatory message. No partial order is written.
3. Create the `Order` with status `pending`.
4. Create one `OrderItem` per cart line, copying the current product name and price.
5. Decrement each product's `stock` by its line quantity.
6. Clear the session cart.
7. Redirect to the order confirmation page.

The transaction is required, not a refinement: without it, a failure partway through can leave an order with missing lines, or stock decremented against an order that was never created.

5.7 Order confirmation — `/order/<id>/`

- Order number, date, status, shipping address, every line item, and the total.
- A note that payment is cash on delivery.
- A user can only open their own orders. Requesting another user's order ID returns 404, not 403, because a 403 confirms that the order exists.

5.8 Order history — `/orders/`

Login required. A table of the user's own orders: order number, date, total, status, and a link to each order's detail page. Newest first.

5.9 Admin panel — `/admin/`

Django Admin, configured as follows:

- **Product:** list shows name, category, price, stock, and `is_active`; price and stock are editable directly in the list; filter by category and `is_active`; search by name; `slug` prefilled from `name`.
- **Category:** list of names, with `slug` prefilled from `name`.
- **Order:** list shows ID, user, total, status, and date; filter by status and date; search by ID and email; `status` is editable so staff can move an order through its lifecycle. Order items appear inline and read-only — a placed order's lines must not be rewritten after the fact.
- **User:** Django's default user admin, unchanged.

---

6. Project Structure

```
e-commerce-store/
+-- manage.py
+-- requirements.txt
+-- .env                    # not committed
+-- .env.example
+-- .gitignore
+-- PRD.md
+-- config/                 # project settings
|   +-- settings.py
|   +-- urls.py
|   +-- wsgi.py
+-- store/                  # products, cart, orders
|   +-- models.py
|   +-- views.py
|   +-- urls.py
|   +-- forms.py
|   +-- admin.py
|   +-- cart.py             # session cart helper
|   +-- migrations/
+-- accounts/               # registration, login, logout
|   +-- views.py
|   +-- urls.py
|   +-- forms.py
+-- templates/
|   +-- base.html
|   +-- accounts/
|   |   +-- login.html
|   |   +-- register.html
|   +-- store/
|       +-- product_list.html
|       +-- product_detail.html
|       +-- cart.html
|       +-- checkout.html
|       +-- order_detail.html
|       +-- order_list.html
+-- static/
|   +-- css/style.css
|   +-- js/cart.js
+-- media/products/         # uploaded images, not committed
```

Two apps: `store` for the catalog, cart, and orders; `accounts` for registration and login. Splitting further would add structure without reducing complexity at this size.

---

7. URL Map

Method	URL	Page	Auth
GET	`/`	Product list	—
GET	`/category/<slug>/`	Products in a category	—
GET	`/product/<slug>/`	Product detail	—
GET	`/cart/`	Cart	—
POST	`/cart/add/<id>/`	Add to cart	—
POST	`/cart/update/<id>/`	Change quantity	—
POST	`/cart/remove/<id>/`	Remove line	—
GET, POST	`/checkout/`	Checkout	Customer

Method	URL	Page	Auth
GET	`/order/<id>/`	Order detail	Owner only
GET	`/orders/`	Order history	Customer
GET, POST	`/accounts/register/`	Registration	Guest
GET, POST	`/accounts/login/`	Login	Guest
POST	`/accounts/logout/`	Logout	Customer
—	`/admin/`	Admin panel	Staff

Every state-changing request is a POST carrying Django's CSRF token.

---

8. Database Configuration

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "localhost"),
        "PORT": os.environ.get("DB_PORT", "5432"),
    }
}
```

Credentials come from environment variables loaded from `.env`. `.env` is listed in `.gitignore`; `.env.example` is committed with placeholder values so a new developer knows which variables to set.

`requirements.txt`:

```
Django>=6.0,<6.1
psycopg[binary]>=3.2
python-dotenv>=1.0
Pillow>=11.0
```

All schema changes go through Django migrations (`makemigrations`, then `migrate`). No hand-written SQL.

---

9. Security Requirements

These are not optional polish. Each one closes a specific hole.

Requirement	Why
Passwords stored only as Django PBKDF2 hashes	Plaintext passwords must never be stored or logged. Django's default hasher does this correctly; do not replace it.
CSRF token on every POST form	Without it, another site can submit forms as the logged-in user.
Django ORM for all queries	Queries are parameterized by default, which prevents SQL injection. Avoid `.raw()` and SQL built by string concatenation.
Template auto-escaping left on	Prevents stored XSS from product descriptions. Never apply the `safe` filter to text entered by users or staff.
`SECRET_KEY`, database password, and `DEBUG` read from the environment	Keeps secrets out of version control.
`DEBUG = False` outside local development	With `DEBUG = True`, an error page exposes settings, environment variables, and SQL to any visitor.

Requirement	Why
Order detail queries filtered by `user=request.user`	Otherwise changing the ID in the URL exposes another customer's name, address, and phone number.
Uploaded images validated as images and capped in size	An unchecked upload path accepts arbitrary files.
Stock re-checked inside the checkout transaction	The cart lives in session data the client can influence, so quantity must never be trusted at checkout.
Price read from the database at checkout, never from the request	Otherwise a crafted request can set its own price.
No card or payment data collected anywhere	Nothing sensitive to leak. This is the strongest argument for mock checkout at this scope.

---

10. Acceptance Criteria

Version 1.0 is done when all of the following pass by hand:

1. The home page lists seeded products with images and prices.
2. Clicking a category filters the list to that category.
3. Clicking a product opens its detail page with the full description.
4. Adding a product to the cart increments the header count and shows the item in the cart.
5. Changing a cart quantity updates the line total and the cart total without a page reload.
6. Removing the last cart item shows the empty-cart message.
7. A new visitor can register and is logged in afterward.
8. Logging out and back in with the same credentials succeeds.
9. A wrong password shows an error and does not log the user in.
10. A guest clicking "Proceed to checkout" lands on login, and after logging in arrives at checkout with the cart intact.
11. Placing an order creates one `Order` row and one `OrderItem` row per cart line, with the correct total.
12. After checkout, each ordered product's stock has dropped by the ordered quantity.
13. After checkout the cart is empty.
14. The confirmation page shows the correct order number, lines, and total.
15. Order history lists the order, newest first.
16. Opening another user's order ID returns 404.
17. Adding a quantity above available stock is rejected, and no order is written.
18. A staff user can create, edit, and delete a product in `/admin/`, and the storefront reflects the change.
19. A staff user can see the placed order in `/admin/` and change its status to `shipped`.
20. A non-staff user visiting `/admin/` is denied access.

---

11. Build Order

Each step leaves the project in a working, runnable state.

Step	Work	Done when
1	Django project, PostgreSQL connection, `.env`	`python manage.py migrate` succeeds
2	`Category` and `Product` models, admin registration, superuser	Products can be added in `/admin/`

Step	Work	Done when
3	`base.html`, `style.css`, product list, category filter	Products render on `/`
4	Product detail page	Detail page reachable from the list
5	Session cart: add, view, update, remove	Items can be added and removed
6	`cart.js` for inline quantity updates and the header count	Totals update without a reload
7	Registration, login, logout	A new account can be created and used
8	`Order` and `OrderItem` models	Migrations applied
9	Checkout inside a transaction, with the stock re-check	An order is placed and stock drops
10	Order confirmation and order history	Orders are visible to their owner only
11	Admin tuning for orders, with read-only inline items	Staff can manage orders
12	Walk through all 20 acceptance criteria	Every item passes

---

12. Open Questions

#	Question	Default if unanswered
Q1	Currency and symbol?	USD, $, two decimal places
Q2	Seed the catalog with sample products?	Yes — a management command creating 3 categories and about 12 products, so the storefront is not empty on first run
Q3	Should a customer be able to cancel a `pending` order?	No — out of scope for 1.0; staff change status in the admin
Q4	Deployment target?	Local development only (`runserver`); production hosting is a later decision